

```

/**
 * server/lib/ai-semaphore.ts
 *
 * Three-tier AI semaphore system.
 *
 * PROBLEM: All callers shared one 8-slot background pool. shorts-pipeline-engine,
 * which needs to prep clips + metadata + SEO + thumbnails before quota resets,
 * kept getting dropped by infinite-evolution, knowledge-mesh, and
 * autonomous-capability-engine filling every slot.
 *
 * SOLUTION: Three independent pools. Pipeline tiers are never blocked by
 * background optimization engines. The publisher at upload time makes ZERO
 * AI calls – it only reads pre-computed rows from the DB.
 *
 * Tier ownership:
 *
 *   shorts_pipeline (3 slots)
 *     → shorts-pipeline-engine   clip scoring, best-moment extraction
 *     → pre-encoder               clip download + ffmpeg encode
 *     → shorts-prep-pipeline      title, description, SEO tags, thumbnail concept
 *
 *   longform_pipeline (2 slots)
 *     → longform-prep-pipeline    title, description, SEO tags, thumbnail concept
 *     → youtube-ai-orchestrator   strategic selection, script analysis
 *     → youtube-back-catalog-runner catalog scoring
 *
 *   background (8 slots)
 *     → infinite-evolution, knowledge-mesh, autonomous-capability-engine,
 *       consistency-agent, auto-thumbnail, content-grinder, repurpose-engine,
 *       vod-seo-optimizer, and every other optimization/improvement engine
 */

```

```

import pLimit from 'p-limit';
import { createLogger } from './logger.js';

```

```

const log = createLogger('ai-semaphore');

```

```

// ——— Tier type —————

```

```

export type AiTier = 'shorts_pipeline' | 'longform_pipeline' | 'background';

```

```

// ——— Pool sizes —————

```

```

export const TIER_LIMITS: Record<AiTier, number> = {
  shorts_pipeline: 3,
  longform_pipeline: 2,
  background: 8,
};

```

// — Three independent pools —

```
const pools: Record<AiTier, ReturnType<typeof pLimit>> = {
  shorts_pipeline:  pLimit(TIER_LIMITS.shorts_pipeline),
  longform_pipeline: pLimit(TIER_LIMITS.longform_pipeline),
  background:       pLimit(TIER_LIMITS.background),
};
```

// — Public API —

```
/**
 * Returns the pLimit instance for the given tier.
 * Wrap async AI calls in this to get semaphore protection.
 *
 *   const sem = getAiSemaphore('shorts_pipeline');
 *   return sem(() => callOpenAI({ messages, tier: 'shorts_pipeline' }));
 */
export function getAiSemaphore(tier: AiTier = 'background') {
  return pools[tier];
}

/**
 * Fast-reject if the tier is at capacity. Call this BEFORE entering the
 * semaphore so callers get an immediate error instead of waiting in queue.
 */
export function assertTierCapacity(tier: AiTier, module: string): void {
  const limit = TIER_LIMITS[tier];
  const pool  = pools[tier];

  if (pool.activeCount >= limit) {
    const label = {
      shorts_pipeline:  'Shorts Pipeline',
      longform_pipeline: 'Long-form Pipeline',
      background:       'background',
    }[tier];

    throw new Error(
      `AI queue full for ${label} tasks ` +
      `(${pool.activeCount}/${limit} active, ${pool.pendingCount} pending) — request dropped`
    );
  }
}

/**
 * Snapshot of all three tier queues.
 * Expose via /api/health or diagnostics panel.
 */
export function getAiQueueStatus() {
  return {
```

```
shorts_pipeline: {
  active: pools.shorts_pipeline.activeCount,
  pending: pools.shorts_pipeline.pendingCount,
  limit: TIER_LIMITS.shorts_pipeline,
},
longform_pipeline: {
  active: pools.longform_pipeline.activeCount,
  pending: pools.longform_pipeline.pendingCount,
  limit: TIER_LIMITS.longform_pipeline,
},
background: {
  active: pools.background.activeCount,
  pending: pools.background.pendingCount,
  limit: TIER_LIMITS.background,
},
};
}
```